

AICL

Architecture Compilation with Auditable Provenance

If the compiler cannot explain why it generated a line of code,
it should not generate it. And this property must be
independently verifiable without trusting the compiler.

Version 2.0.0
Philippe-Antoine
June 2026

MIT License

Table of Contents

Placeholder for table of contents	0
-----------------------------------	---

1. Abstract

This paper presents AICL (Architecture Compilation Language), a specification-first programming system where risks, recoveries, and validations are mandatory syntactic elements, and every generated line of code carries full provenance. The central thesis of this work is the principle of **explicable compilation**: a compiler that cannot explain why it generated a line of code should not generate it.

Unlike conventional compilers that treat compilation as a pure translation from source to target, AICL treats compilation as a reasoning process where architecture, risks, recoveries, and validations all contribute to the generated code and are traceable back to their origin. The system enforces a deterministic compilation contract: the same input always produces the same output, with no probabilistic or AI-driven steps in the core compilation pipeline. AI assistance is available as an optional plugin, strictly separated from the deterministic compiler.

The language provides 27 reserved keywords across 10 progressive levels, a 9-stage compilation pipeline, 30+ deterministic behavior patterns, and a provenance tracker that records the reasoning path for every compilation decision. This paper describes the language design, compilation architecture, provenance system, and the philosophical foundation of explicable compilation.

2. The Problem: Silent Compilers

Every production software system requires error handling, failure recovery, and validation. In every mainstream programming language, these concerns are optional afterthoughts. They are scattered across try/catch blocks, buried in documentation, or forgotten entirely. The gap between what architects intend to build and what developers actually build is where most production failures live.

But there is a deeper problem than the absence of risk management in source code. Modern compilers generate millions of lines of code without ever being able to answer a simple question: **why?** Why was this line generated? What architectural decision produced it? What risk does this recovery address? What validation does this test verify? No mainstream compiler can answer these questions. The code exists, but the reasoning behind it is lost the moment compilation finishes.

This loss of reasoning has practical consequences. When a production system fails at 3:00 AM, the on-call engineer must reconstruct the intent behind code they did not write, based on error messages that reference code whose purpose is undocumented. Architecture documents describe risk scenarios and recovery strategies, but nothing enforces their implementation. Code reviews miss edge cases. Tests skip failure paths. The result: systems that work on the happy path and crumble on every other one.

The rise of AI-generated code has made this problem worse, not better. AI code generators produce code that works, sometimes, for some inputs, but they cannot tell you why they chose one implementation over another, whether they considered the edge cases your architecture document describes, or what risks they failed to address. The compiler has become a black box, and the black box is getting blacker.

3. The Thesis: Explicable Compilation

If the compiler cannot explain why it generated a line of code, it should not generate it.

This principle, which we call **explicable compilation**, is the central thesis of AICL. It is not a debugging feature or an after-the-fact documentation tool. It is a design constraint on the compiler itself: the compiler must be structured such that every output line has a traceable provenance chain back to the source specification.

3.1 What Explicable Compilation Means

Explicable compilation means three things simultaneously. First, **provenance**: every generated line must be traceable to a specific source construct, a specific compilation decision, and a specific transformation step. Second, **determinism**: the compilation process must be fully deterministic so that the provenance chain is reproducible. The same input always produces the same output with the same provenance records. Third, **accountability**: if the compiler cannot produce a provenance chain for a generated line, it must fall back to an explicit sub-language syntax rather than guessing. The compiler would rather acknowledge its limitations than silently guess.

3.2 What Explicable Compilation Does Not Mean

Explicable compilation does not mean that the compiler must generate human-readable explanations for every line of code as comments. It does not mean that the compiler must use AI to justify its decisions. And it does not mean that the generated code must be trivially simple so that it is easy to explain. Rather, it means that the compiler must maintain an internal audit trail that can be queried and inspected, and that this audit trail must be the primary artifact of compilation, not a secondary byproduct.

The distinction is important. A conventional compiler could, in theory, be modified to emit annotations about its optimization passes. But these annotations would be post-hoc rationalizations of decisions already made. In AICL, the provenance record is produced before the code is generated: the compiler first determines what it will generate and why, records this decision, and then generates the code. The provenance is not a commentary on the code; it is the reason for the code.

3.3 The Provenance Principle

The provenance principle can be stated formally: for every line *L* in the generated output, there must exist a provenance record *P* such that *P.source* maps to a construct in the AICL input, *P.resolution_path* describes the sequence of compilation decisions that led to *L*, *P.confidence* quantifies the compiler's certainty in the match, and *P.generated_code* contains *L*. If no such *P* can be constructed, the compiler must not generate *L*; instead, it must use an explicit fallback mechanism that the programmer can inspect and override.

This principle has profound implications for compiler design. It means that the compiler cannot silently fill in code based on heuristics without recording those heuristics. It cannot optimize code in ways that are not traceable. And it cannot use probabilistic methods (including AI) in the core compilation path without violating

the determinism requirement that makes provenance reproducible.

4. Language Design

AICL is a specification-first language where the programmer describes what a system must achieve rather than how it achieves it. The language is organized in 10 progressive levels, where Level 1 (Architecture) is mandatory for all programs and higher levels add capabilities as needed. This design ensures that even the simplest AICL program carries the minimal architectural information needed for explicable compilation: a goal, a layer structure, and at least one validation criterion.

4.1 Core Principles

Three principles govern the language design. **Architecture as program**: the architectural specification is the program, not a document about the program. Layers, entities, and behaviors are not documentation; they are compilation inputs that directly determine the structure of the generated code. **Risk as syntax**: risks and recoveries are not optional error-handling patterns; they are mandatory syntactic elements that the compiler enforces the way other compilers enforce type systems. **Validation as test**: validation sections are not aspirational statements; they are compiled into executable test code that verifies the system meets its specification.

4.2 The Ten Language Levels

Level	Feature	Keywords	Purpose
1	Architecture	Goal, Constraint, Risk, Recovery, Layer, Sublayer, Validation	Mandatory
2	Entities	Entity + typed fields	Data models
3	Behaviors	Behavior, Input, Output, Action	Entity operations
4	Conditions	Condition, When, Then	Decision logic
5	Events	Event, On, Action	Event-driven logic
6	Concurrency	Parallel	Concurrent execution
7	Optimization	Optimize, Priority	Performance targets
8	Learning	Learn, Adapt, Based	ML integration
9	Security	Security, Encrypt, Protect	Security directives
10	Native Code	Native	Escape hatch

Table 1: AICL Language Levels

4.3 Minimal Valid Program

The smallest meaningful AICL program requires only three constructs: a Goal that states what the system must achieve, a Layer that defines a structural component, and a Validation that specifies how to verify correctness. This minimal program is already richer than the equivalent hello-world in any mainstream language because it encodes intent, structure, and a test criterion in four lines:

```
Goal:
    Hello World
Layer:
    Core
Validation:
    Output is produced
```

4.4 Risk and Recovery as Mandatory Constructs

In AICL, every Risk must have a paired Recovery. This is not a style guideline or a linting rule; it is a compile-time requirement enforced by the grammar. The compiler pairs risks with recoveries during the Risk Analysis stage (Stage 4) and synthesizes recovery logic during Recovery Synthesis (Stage 5). If a Risk has no Recovery, the compiler generates a generic fallback recovery and records a provenance entry indicating that the recovery was synthesized rather than specified.

This design ensures that production systems built with AICL cannot silently ignore failure modes. The architecture document that describes risks and recovery strategies is no longer a separate artifact that may or may not be implemented; it is the source code itself, and the compiler guarantees that every identified risk has corresponding recovery logic in the generated output.

```
Risk:
    Ball leaves play area
Recovery:
    Clamp ball position to boundaries
```

The compiler transforms this risk/recovery pair into executable code that clamps the ball position, and the provenance record traces this code back to the specific Risk and Recovery sections in the source. When a developer runs `aicl explain`, they can see exactly why the clamping code exists and what failure scenario it addresses.

4.5 Validation as Compiled Tests

Validation sections in AICL are not comments about what should be tested. They are compiled into executable pytest suites during the Test Generation stage (Stage 7). The compiler uses keyword analysis to determine what each validation checks and generates appropriate test methods. For example, the validation 'Complete one playable match' generates a test that verifies the game loop executes successfully, while 'Maintain 60 FPS during gameplay' generates a performance test.

The provenance system tracks each generated test back to its source Validation section, creating a bidirectional link between specification and verification. This means that when a test fails, the developer can immediately identify which architectural requirement is not being met, rather than reverse-engineering the test's intent from its implementation.

5. Compilation Pipeline

The AICL compiler transforms architectural specifications into executable source code through a 9-stage pipeline. Unlike conventional compilers that focus primarily on syntactic and semantic analysis followed by code generation, the AICL pipeline treats risk analysis, recovery synthesis, and provenance recording as first-class compilation stages. Each stage produces intermediate artifacts and provenance records that are carried forward through subsequent stages.

Stage	Name	Function
1	Parsing	Source to AST with 17 node types
2	Architecture Validation	Structural completeness (Goal, Layer, Validation present)
3	Dependency Analysis	Layer dependency graph and execution order
4	Risk Analysis	Risk/Recovery pairing (every Risk needs a Recovery)
5	Recovery Synthesis	Recovery logic generation with provenance recording
6	Code Generation	Python source with 30+ deterministic behavior patterns
7	Test Generation	pytest suite from Validation sections
8	Optimization	Apply Optimize/Priority hints
9	Final Construction	Assemble main.py + test_main.py + architecture tree

Table 2: The 9-Stage Compilation Pipeline

5.1 Deterministic Behavior Compilation

The key innovation in the code generation stage is the Behavior Pattern Library: a set of 30+ deterministic patterns that map action descriptions to concrete, executable code. When the compiler encounters a Behavior section, it analyzes the action description using keyword-based scoring with word-boundary matching. If a pattern matches with confidence above 0.3, the compiler uses that pattern's code template. If no pattern matches, the compiler falls back to the sub-language parser, which handles explicit action specifications such as 'assign', 'clamp', 'check', 'send', 'return', and 'log'.

This approach guarantees deterministic compilation: the same AICL source always produces the same Python output, regardless of the execution environment, random seed, or any other non-deterministic factor. The compiler does not guess. It either finds a pattern that matches with documented confidence, uses an explicit sub-language specification, or produces a structured fallback skeleton with full provenance indicating that human review is needed.

Category	Patterns
Movement	MOVE, MOVE_BALL, REFLECT_VELOCITY, CLAMP_POSITION
Creation	CREATE, INIT_LAYER, CREATE_WINDOW
Communication	BROADCAST, SEND_MESSAGE

Category	Patterns
Update	UPDATE, INCREMENT_SCORE, UPDATE_STATE, ADD_TO_COLLECTION
Display	DISPLAY, RENDER_FRAME, HIGHLIGHT
Validation	VALIDATE, CHECK_COLLISION
Networking	CONNECT, RECONNECT, DISCONNECT
Persistence	STORE, LOAD
Security	ENCRYPT_DATA, PROTECT_ACCESS
Sync	SYNC_STATE, BUFFER_INPUT
Game	END_GAME, APPLY_PHYSICS
Adaptation	ADAPT_QUALITY, SUGGEST, QUEUE_MESSAGE

Table 3: Behavior Pattern Categories

5.2 Architecture Template Mapping

The compiler uses the Goal section to select an application structure template. The `ArchitectureTemplateMapper` maps goal keywords to one of four structural patterns: `game_loop` (init-update-render-loop, for games and simulations), `event_driven` (connect-listen-handle-respond, for servers and chat systems), `pipeline` (ingest-process-output, for data processing), and `crud` (list-create-read-update-delete, for data management). This template determines the main application loop structure, the method signatures, and the execution model of the generated code.

The template selection is deterministic and provenance-tracked. The compiler records which template was selected, why (based on which keywords in the Goal section), and with what confidence. This means that when a developer questions why their chat application uses an event-driven architecture, the provenance record can answer: because the Goal contained 'chat' and 'server' keywords, which matched the `event_driven` template with confidence 0.87.

5.3 Sub-Language Parser

When no behavior pattern matches with sufficient confidence, the sub-language parser provides an explicit syntax for specifying actions. The sub-language supports eight statement types: `assign` (with operators `+=`, `-=`, `=`, `*=`), `clamp` (between min and max), `check` (with comparison operators), `send` (message to destination), `return`, `call` (method invocation), `log` (message), and `raise` (exception). This sub-language ensures that even when the pattern library does not cover a specific action, the programmer can still express it deterministically without resorting to native code.

```
Behavior MovePaddle
  Input: Player direction string
  Action: assign paddle_position += direction * speed
         clamp paddle_position between 0 and screen_width
```


6. Provenance System

The provenance system is the mechanism through which AICL implements explicable compilation. It records a trace of every compilation decision, from the initial parsing of source constructs to the final generation of code and tests. This trace is not generated after compilation as a debugging aid; it is produced during compilation as an integral part of the compilation process. In v0.5, the provenance system was extended with artifact tracking and audit coverage computation. In v0.7, it became the foundation for the Proof of Origin: a self-contained, independently verifiable compilation artifact.

6.1 Provenance Types

Every compilation decision is classified into one of twenty-three provenance types, each indicating the nature of the decision and the reasoning path that was followed. These types form a taxonomy of compilation reasoning that enables systematic analysis of compiler behavior. The initial twelve types were expanded in v0.5 to cover all code generation zones, and in v2.0 to cover autonomous compilation, AI-assisted generation, pattern learning, and specification evolution, ensuring that every generated artifact has at least one provenance record explaining its existence.

Type	Meaning
PATTERN_MATCH	A behavior pattern matched with sufficient confidence
SUB_LANGUAGE	Explicit sub-language syntax was used
FALLBACK	No pattern matched; structured skeleton generated
ARCHITECTURE_TEMPLATE	Goal mapped to application structure
DIRECT_MAPPING	Keyword directly mapped to code
RECOVERY_SYNTHESIS	Recovery code synthesized from Risk/Recovery pair
VALIDATION_SYNTHESIS	Test code generated from Validation section
CONDITION_SYNTHESIS	Condition handler generated from When/Then
EVENT_SYNTHESIS	Event handler generated from On/Action
ENTITY_GENERATION	Dataclass generated from Entity definition
LAYER_INITIALIZATION	Component initialization from Layer definition
HELPER_METHOD	Utility method generated to support other code
SECURITY_METHOD	Security enforcement method generated
PARALLEL_EXECUTION	Parallel execution code generated
RUN_METHOD	Main run method generated from architecture template
IMPORT_GENERATION	Import statements generated from program analysis
ENTRY_POINT	Main entry point function generated

Type	Meaning
TEST_GENERATION	Test functions generated from specifications
CLASS_STRUCTURE	Class structure (attributes, constructor) generated
AI_GENERATION	Code generated by AI-assisted fallback
SELF_HEALING	Code fixed by autonomous diagnosis
PATTERN_LEARNED	New pattern learned from unmatched actions
SPEC_EVOLVED	Specification modified by autonomous evolution

Table 4: Provenance Types (23 types, full coverage)

6.2 The Provenance Record

Each provenance record contains five core fields: **source**, which identifies the AICL construct that triggered the compilation decision; **resolution_path**, which describes the sequence of steps the compiler took to arrive at the decision; **generated_code**, which contains the actual code that was produced; **confidence**, which quantifies the compiler's certainty in the match (for pattern-based decisions) or is set to 1.0 (for deterministic mappings); and **pattern_name**, which identifies the specific pattern or mechanism that was used. Additionally, each record carries **artifact_names**, which links the record to the specific generated artifacts it explains.

The `aicl explain` command traverses these provenance records and produces a human-readable trace of the compilation process. When invoked with the `--behavior` flag, it focuses on a specific behavior and shows the pattern matching process in detail. When invoked with `--proof`, it reads from the Proof of Origin file directly, without requiring the compiler.

6.3 The Auditability Contract

The provenance system enforces an auditability contract: every generated artifact must have at least one provenance record. If the compiler generates an artifact without a corresponding provenance record, this is an **orphan artifact** — a compiler bug, not an acceptable gap. This contract ensures that the `aicl audit` command can always provide a complete accounting of the compilation process.

The auditability contract also serves as a constraint on future compiler development. Any new optimization pass, code generation strategy, or transformation must be accompanied by provenance recording. This prevents the compiler from growing features that would violate the explicable compilation principle. The `aicl audit --strict` command enforces this contract: it returns a non-zero exit code if audit coverage is below 100%, making it suitable for CI/CD pipelines.

6.4 Audit Coverage

Audit coverage is the key metric of the auditability contract. It measures the fraction of generated artifacts that have at least one provenance chain linking them to their originating specification. Formally:

$$\text{Audit Coverage} = |\{A \text{ in Artifacts} : A \text{ has provenance chain}\}| / |\text{Artifacts}|$$

The target is always Audit Coverage = 1.0 (100%). In the current implementation, all four example programs achieve 100% audit coverage with zero orphan artifacts. This is not an accident but a direct consequence of the v0.5 expansion of provenance types to cover all code generation zones. Before v0.5, certain code generation points (security methods, import blocks, entry points, test functions, class structures) did not record provenance, creating orphan artifacts. The v0.5 changes ensured that every generation point in the compiler records provenance, making 100% audit coverage the default rather than the exception.

7. Deterministic Compilation Contract

AICL enforces a strict deterministic compilation contract: the same AICL source code, compiled with the same compiler version and target language, must always produce identical output. This is not merely a desirable property; it is a fundamental design requirement that enables the provenance system to provide meaningful and reproducible explanations.

The determinism contract has several implications. First, the compiler cannot use random number generation, current timestamps, or any other non-deterministic input in the compilation path. Second, the compiler cannot use AI models in the core pipeline, since AI model outputs are inherently probabilistic. Third, the behavior pattern matching algorithm must be deterministic: given the same input, it must always select the same pattern with the same confidence score.

AI assistance is available in AICL through the `--ai-fill` flag, but this mode is explicitly separate from the core compiler. When AI fill is used, the provenance records clearly indicate which code was generated by AI and which was generated deterministically. This separation ensures that developers always know which parts of their codebase have deterministic provenance and which parts rely on probabilistic generation.

8. Proof of Origin

The Proof of Origin is the central artifact of AICL's auditable compilation system. Introduced in v0.6 and made fully self-contained in v0.7, it represents a fundamental architectural shift in how compilation outputs are structured. Rather than producing code alone, or code with internal provenance data, the compiler now produces a single self-contained proof artifact from which code, explanations, and audit reports can all be reconstructed.

8.1 The Architectural Shift

The evolution of AICL's output architecture reveals a deepening understanding of what auditable compilation means. In the beginning, AICL produced code: a traditional compiler output with no accountability mechanism. Then came explain: a human-readable trace of why each line was generated, but one that required running the

compiler again. Then came audit: a verification that every artifact has provenance, but again requiring the compiler.

The Proof of Origin collapses these into a single architecture. The compiler produces the proof, and from the proof, everything else follows. Code is in the proof. Explain is a view on the proof. Audit is a verification of the proof. The proof becomes the central artifact, and the compiler becomes just the mechanism that produces it.

```
AICL source --> Compiler --> Proof of Origin (.aicl-proof)
|
+-- Code (extracted from proof)
+-- Explain (reconstructed from proof)
+-- Audit (verified from proof)
```

Figure 2: The Proof of Origin as Central Artifact

8.2 Self-Contained Verification (Format v2.0)

The critical design question for the Proof of Origin was: if `aicl explain` were deleted entirely, would the `.aicl-proof` file contain enough information to reconstruct the full explanation? In format v1.0, the answer was no: the proof contained hashes and provenance records but not the generated code itself. An external verifier could check structural properties but could not verify hash binding without separately obtaining the code.

Format v2.0, introduced in v0.7, makes the proof fully self-contained by embedding the original AICL source text, the generated program code, and the generated test code directly in the proof file. This means an independent verifier can: (1) recompute SHA-256 hashes and compare them to the stored hashes, (2) verify that the proof is cryptographically bound to specific code artifacts, and (3) reconstruct complete explanations and audit reports from the proof alone, without needing the AICL compiler, parser, or any project dependency.

8.3 The Independent Verifier

The existence of a self-contained proof file raises an immediate question: who verifies the verifier? If the only tool that can verify a proof is AICL itself, the system collapses into circular reasoning: *AICL says AICL is correct*. This is the "trust me bro" problem, and it is a serious threat to the credibility of any self-auditing system.

To address this, AICL v0.7 includes an independent verifier: `tools/verify_proof.py`, approximately 200 lines of Python that use only the standard library (`json`, `hashlib`, `sys`). This verifier has zero dependencies on AICL: no compiler, no parser, no project modules. It reads the `.aicl-proof` file as plain JSON and applies eight deterministic verification checks:

Check	Description
<code>format_version</code>	Proof format version is supported
<code>source_hash_binding</code>	SHA-256(<code>source_text</code>) matches <code>source_hash</code>
<code>program_hash_binding</code>	SHA-256(<code>generated_source</code>) matches <code>program_hash</code>
<code>test_hash_binding</code>	SHA-256(<code>generated_tests</code>) matches <code>test_hash</code>

Check	Description
no_orphan_artifact_property	Every artifact has at least one provenance chain
complete_coverage_property	Audit Coverage = 1.0
record_artifact_linkage	All provenance indices reference valid records
artifact_consistency	Orphan status consistent with provenance linkage

Table 5: Independent Verifier Checks (8 checks, zero AICL dependencies)

The verifier can be run by anyone, anywhere, without installing AICL. It provides a clear binary verdict: VALID or INVALID. If the proof passes all eight checks, the No-Orphan Property is verified as a property of the artifact itself, not as a claim of the compiler. This is the critical distinction: the property is not a feature of the tool that produced the artifact; it is a verifiable characteristic of the artifact, checkable by any third party with a 200-line script.

8.4 Hash Binding

The Proof of Origin uses SHA-256 hashes to cryptographically bind the proof to the generated code. Three hashes are stored: **source_hash** (SHA-256 of the original AICL source text), **program_hash** (SHA-256 of the generated program), and **test_hash** (SHA-256 of the generated tests). In format v2.0, the verifier recomputes these hashes from the embedded code and compares them to the stored values. Any tampering with the generated code, the source text, or the proof file itself will be detected as a hash mismatch, causing verification to fail.

This hash binding creates a chain of trust: the source hash ties the proof to the original specification, the program hash ties the proof to the generated code, and the test hash ties the proof to the test suite. Any break in this chain invalidates the entire proof.

9. Formal Properties

AICL's auditable compilation system establishes three formal properties that are verified in every compilation. These are not aspirational goals but checkable properties: each one is tested by the independent verifier and must pass for a Proof of Origin to be considered valid.

9.1 No Orphan Artifact Property

For any compilation C, if artifact A is generated, then A has at least one provenance chain linking it to its originating specification.

This is the foundational property of auditable compilation. An orphan artifact is a generated piece of code (method, class, function, or test) that exists in the output but has no provenance record explaining why it was generated. The No Orphan Property states that such artifacts must not exist. In the current implementation, all four example programs satisfy this property with zero orphan artifacts across a total of 204 generated artifacts.

The significance of this property extends beyond AICL. The question it raises is universal: should any code-generating system — whether a traditional compiler, an AI code generator, or a model-driven architecture tool — be able to demonstrate that every line of code it produces has a traceable origin? The No Orphan Property provides a formal framework for answering this question.

9.2 Complete Coverage Property

$$\text{Coverage} = |\{A \text{ in Artifacts} : A \text{ has provenance chain}\}| / |\text{Artifacts}| = 1.0$$

The Complete Coverage Property is the quantitative expression of the No Orphan Property. It states that the fraction of artifacts with provenance must equal exactly 1.0. Anything less means at least one artifact lacks justification. The independent verifier computes this ratio independently from the proof file's artifact list and provenance records, then compares it to the declared value. Both must equal 1.0 for the proof to be valid.

9.3 Hash Binding Property

$$\text{SHA-256}(\text{generated_code}) = \text{program_hash stored in Proof of Origin}$$

The Hash Binding Property ensures that the proof is cryptographically bound to the generated code. Without this property, a proof file could be detached from its code artifact and attached to different code, defeating the purpose of provenance tracking. The SHA-256 hash creates a tamper-evident link: any modification to the generated code, the source text, or the proof itself will cause a hash mismatch, immediately invalidating the proof.

10. Formal Grammar

AICL has a formal BNF grammar with 27 reserved keywords. The grammar is line-based, supporting two formats for keyword-value pairs: inline (Keyword: Value on the same line) and block (Keyword: on one line, indented Value on the next). This dual format allows both concise single-line specifications and multi-line descriptions, making the language readable for both simple and complex programs.

The complete BNF grammar is provided in the specification file at `spec/grammar.md`. The grammar defines productions for all 10 language levels, including nested structures such as Entity fields within Entity blocks, Behavior sub-sections (Input, Output, Action), and indented Security action blocks. The grammar enforces the Risk/Recovery pairing requirement at the structural level.

10.1 Type System

AICL provides 11 built-in type names that can be used in Entity field declarations and Behavior input/output specifications: string, integer, float, boolean, datetime, list, dict, set, any, void, and bytes. The type system is intentionally simple: it provides enough expressiveness for architectural specification without introducing the

complexity of a full type inference system. Types are used for documentation and code generation purposes; they do not carry runtime type-checking semantics in the current implementation.

10.2 Keyword Reference

Keyword	Level	Purpose
Goal	1	Define system objective
Constraint	1	Specify system constraints
Risk	1	Identify potential failure
Recovery	1	Define failure response
Layer	1	Define architectural layer
Sublayer	1	Define sub-component
Validation	1	Specify verification criterion
Entity	2	Define data model
Behavior	3	Define operation
Input	3	Define behavior input
Output	3	Define behavior output
Action	3	Define behavior action
Condition	4	Define conditional logic
When	4	Condition trigger
Then	4	Condition response
Event	5	Define event handler
On	5	Event trigger
Parallel	6	Define concurrent execution
Optimize	7	Define optimization target
Priority	7	Set optimization priority
Learn	8	Define learning objective
Adapt	8	Define adaptation rule
Based	8	Adaptation basis
Security	9	Define security directive
Encrypt	9	Encryption action
Protect	9	Protection action
Native	10	Embed native code

Table 7: Complete Keyword Reference

11. Intermediate Representation

Between the AST and the code generation stage, the compiler constructs an Architecture Tree: a hierarchical intermediate representation that organizes the flat list of AST sections into a tree structure reflecting the system's architectural decomposition. The root node represents the System as a whole, with child nodes for each Layer, Entity, and Behavior. Risks, recoveries, validations, and constraints are attached to the appropriate nodes in the tree.

The Architecture Tree serves several purposes. It provides a structural basis for the dependency analysis stage, which determines the order in which layers should be initialized and executed. It enables risk distribution, which attempts to associate each risk with the architectural layer most likely to be affected. And it provides a visualization of the system architecture that can be output as a text file using the `aicl tree` command.

The tree visualization uses symbolic annotations to indicate the presence of risks, recoveries, validations, and constraints at each level. The symbol set includes warning markers for risks, recovery markers for recovery strategies, check markers for validations, and constraint markers for architectural constraints. This provides a quick visual summary of where risk management and verification are concentrated in the architecture.

12. Comparative Analysis

AICL occupies a unique position in the landscape of programming languages, domain-specific languages, and code generation systems. This section compares AICL with related approaches along the dimensions that matter most for explicable compilation.

Feature	Mainstream	AI Code Gen	Traditional DSL	AICL
Risk/Recovery mandatory	No	No	No	Yes
Validation compiles to tests	No	No	No	Yes
Full compilation provenance	No	No	No	Yes
Deterministic compilation	Yes	No	Yes	Yes
Architecture as program	No	Partial	No	Yes
AI-free core pipeline	Yes	No	Yes	Yes
Specification-first syntax	No	No	Yes	Yes
Proof of Origin artifact	No	No	No	Yes
Independent verification	No	No	No	Yes
No Orphan Property (formal)	No	No	No	Yes

Table 8: Comparative Analysis

12.1 Mainstream Languages

Languages like Python, Rust, Go, and C++ treat error handling as an optional practice. Even Rust, which enforces error handling through its Result type, does not require developers to identify risks before writing code or to pair each risk with a recovery strategy. None of these languages provide compilation provenance. The compiler generates code but cannot explain why specific code was generated or trace generated lines back to architectural decisions.

12.2 AI Code Generation

AI code generators like GitHub Copilot, ChatGPT, and Cursor produce code that is often functional but never explainable in the AICL sense. These systems use probabilistic models to generate code, meaning the same prompt may produce different code on different runs. There is no provenance chain linking generated code to the architectural intent that prompted it. The code works until it does not, and when it fails, there is no trace of the reasoning that produced it. AICL's deterministic compilation contract and provenance system are direct responses to this problem.

12.3 Traditional DSLs and ADLs

Domain-specific languages and architecture description languages share AICL's specification-first philosophy. However, traditional DSLs focus on generating configuration, boilerplate, or specialized code within a narrow domain. They do not treat risks as mandatory constructs, do not compile validations into tests, and do not provide compilation provenance. Architecture description languages like ArchiMate and SysML describe system structure but do not compile into executable code with error handling. AICL bridges the gap between architectural specification and executable code while maintaining the provenance chain that makes the bridge traceable.

13. Autonomous Compilation and Self-Writing Code

The principle of explicable compilation — that the compiler can explain why it generated every line of code — has a logical extension: if the compiler can explain why it generated something, it can also explain why it needs to fix something, why a pattern should be learned, and why a specification should evolve. Version 2.0 of AICL introduces autonomous compilation: a self-writing, self-validating loop that applies the same provenance-tracking discipline to the act of modification as the original compiler applied to the act of generation.

13.1 The Autonomous Compilation Loop

The autonomous compilation system implements a self-writing, self-validating loop that iterates until convergence. The loop follows the sequence:

```
SPEC &rarr; COMPILE &rarr; VERIFY &rarr; TEST &rarr; DIAGNOSE &rarr; FIX &rarr;
RECOMPILE
```

Each iteration of the loop preserves the No-Orphan Property: every line of code generated or modified during the autonomous process carries full provenance. The convergence criteria are strict and all must be satisfied simultaneously: audit coverage = 100%, all tests pass, zero fallback artifacts remain, the specification is verified as complete and consistent, and the delta between successive iterations falls below a configurable threshold.

This is the logical extension of explicable compilation: if the compiler can explain why it generated every line, it can also explain why it needs to fix a line. The diagnostic stage produces a provenance record for each identified issue, and the fix stage produces a provenance record for each correction. The autonomous loop is not a black box that mysteriously improves code; it is a transparent, provenance-tracked process where every modification has a documented reason.

13.2 Pattern Learning

The PatternLearner extracts new behavior patterns from unmatched action descriptions. When the compiler encounters an action that does not match any existing pattern in the behavior pattern library, the PatternLearner analyzes the description to extract a verb (the operation) and an object (the target), classifies the pair into a category, and generates a code template for that category.

Verb extraction recognizes 40+ action verbs including create, update, validate, compute, transform, store, load, encrypt, connect, notify, delete, check, process, handle, generate, calculate, convert, format, parse, filter, sort, merge, split, encrypt, authenticate, authorize, log, monitor, schedule, deploy, register, configure, initialize, reset, refresh, sync, backup, restore, archive, and migrate.

Object extraction identifies both domain-specific and general objects. Financial objects include account, transaction, card, loan, payment, balance, currency, and invoice. General objects include user, message, request, response, session, token, record, event, notification, and report.

Category	Example Verbs
CREATION	create, initialize, generate, register
DELETION	delete, remove, archive, purge
UPDATE	update, modify, set, reset, refresh
VALIDATE	validate, check, verify, authenticate, authorize
ROUTE	route, dispatch, redirect, forward
COMPUTE	compute, calculate, process, aggregate
TRANSFORM	transform, convert, format, parse, encode
STORE	store, save, persist, backup, archive

Category	Example Verbs
LOAD	load, fetch, retrieve, read, restore
ENCRYPT	encrypt, decrypt, hash, sign, seal
CONNECT	connect, link, bind, associate, sync
NOTIFY	notify, alert, broadcast, emit, log
FINANCIAL	charge, refund, transfer, withdraw, deposit
SECURITY	authenticate, authorize, audit, lockdown, revoke
AUDIT	audit, inspect, review, trace, verify
SCHEDULE	schedule, queue, delay, retry, timeout

Table 9: Pattern Learning Categories

Learned patterns start at confidence 0.7, distinguishing them from built-in deterministic patterns which operate at confidence 0.95. This confidence gap ensures that the compiler always prefers established patterns over newly learned ones, while still enabling the system to grow its capability over time. Each learned pattern is tracked with `ProvenanceType.PATTERN_LEARNED`, maintaining the full provenance chain even as the pattern library expands.

13.3 Specification Evolution

The SpecEvolver fixes AICL specifications based on three categories of issues discovered during the autonomous compilation loop:

Verification failures: When the verification stage detects that the specification is incomplete or inconsistent — for example, missing risk/recovery pairs or missing actions for declared behaviors — the SpecEvolver adds the required constructs to the specification.

Test failures: When the generated code fails tests — for example, due to missing imports, undefined attributes, or incomplete definitions — the SpecEvolver modifies the specification to ensure that the next compilation iteration will generate correct code.

Audit gaps: When audit coverage falls below 1.0, indicating that some generated artifacts lack provenance chains, the SpecEvolver adjusts the specification to ensure that all code generation paths are covered by provenance records.

All spec changes are tracked with `ProvenanceType.SPEC_EVOLVED`, maintaining the provenance chain even as the specification itself evolves. This is a critical property: the specification is not a static input but a living document that adapts to ensure compilation success, and every adaptation is recorded and auditable.

13.4 AI-Powered Self-Writing Compilation

The SelfWritingCompiler is the highest level of the autonomous compilation system. It extends the autonomous loop with AI assistance, following the sequence:

```
DESCRIBE &rrarr; AI-GENERATE &rrarr; COMPILE &rrarr; VERIFY &rrarr; TEST &rrarr;
AI-DIAGNOSE &rrarr; AI-FIX &rrarr; RECOMPILE
```

The system takes natural language task descriptions as input — for example, "Build a hospital management system" — and generates complete AICL specifications from scratch. The AICLGenerator component translates the natural language description into a valid AICL specification with appropriate entities, behaviors, conditions, events, risks, recoveries, and validations. The AIDiagnoser component performs AI-powered root cause analysis when the compilation loop encounters failures, proposing fixes that are then applied and verified.

If AI is unavailable — for example, in air-gapped environments or when API limits are reached — the system falls back to the deterministic autonomous loop, which uses PatternLearner and SpecEvolver without AI assistance. This ensures that the self-writing capability degrades gracefully rather than failing entirely.

New CLI commands support the self-writing workflow: `aicl create "Build X"` generates a complete AICL specification from a natural language description, and `aicl evolve source.aicl` runs the autonomous compilation loop on an existing specification. Every AI-generated artifact is tracked with `ProvenanceType.AI_GENERATION`, ensuring that the provenance chain remains complete even when parts of the code are written by AI.

14. Implementation Status

The current implementation (v2.0.0) includes a complete parser for all 10 language levels, a 9-stage compilation pipeline that generates Python code, a behavior pattern library with 30+ deterministic patterns, a sub-language parser for explicit action specifications, a provenance tracker with 23 types that records every compilation decision, an artifact tracking system that registers every generated code unit, an audit system that verifies 100% provenance coverage, a Proof of Origin system that produces self-contained, cryptographically bound proof files, an independent verifier that can validate proofs without any AICL dependency, an autonomous compilation system with pattern learning and specification evolution, and an AI-powered self-writing compiler. The test suite contains 151 tests, all passing. Five example programs of increasing complexity compile to zero-TODO Python code, each achieving 100% audit coverage and producing valid Proofs of Origin.

The compilation pipeline consists of nine stages: Specification Parsing, Architecture Validation, Dependency Analysis, Risk Analysis, Recovery Synthesis, Code Generation (with full provenance and artifact tracking), Test Generation (with provenance and artifact tracking), Optimization, and Final Construction. Each stage records provenance for every decision it makes, ensuring complete traceability from source to output.

The CLI provides 13 commands: `compile`, `parse`, `tree`, `check`, `explain`, `audit`, `proof`, `verify`, `optimize`, `evolve`, `create`, `ai-fix`, `tui`, and `version`. The `compile` command produces Proof of Origin output. The `explain` and `audit` commands can operate entirely from the Proof of Origin file, demonstrating that the proof contains all information needed to reconstruct explanations and audit reports. The `create` and `evolve` commands support the autonomous compilation workflow. The `ai-fix` command invokes AI-powered diagnosis and repair. The `tui` command provides a terminal user interface for interactive exploration.

Metric	Value
Tests	151

Metric	Value
Source code	~13,700 lines
Provenance types	23
Target languages	4 (Python, Rust, JavaScript, Go)
Example programs	5
CLI commands	13
TUI	Terminal interface for interactive exploration

Table 10: Implementation Statistics

15. Future Directions

15.1 Autonomous Compilation

The autonomous compilation system implements a self-writing, self-validating loop that represents the logical extension of explicable compilation. The loop follows the sequence:

```
SPEC &rarr; COMPILE &rarr; VERIFY &rarr; TEST &rarr; DIAGNOSE &rarr; FIX &rarr;
RECOMPILE
```

Convergence is achieved when all of the following criteria are simultaneously satisfied: audit coverage = 100%, all tests pass, zero fallback artifacts remain, the specification is verified as complete and consistent, and the delta between successive iterations falls below a configurable threshold. The key components are:

PatternLearner: extracts verbs and objects from unmatched action descriptions, classifies them into categories, and generates code templates. Learned patterns start at confidence 0.7 (vs deterministic patterns at 0.95), ensuring the compiler always prefers established patterns over newly learned ones.

SpecEvolver: fixes verification failures (missing risk/recovery pairs, missing actions), test failures (missing imports, attributes, definitions), and audit gaps (coverage < 1.0).

TestRunner: runs pytest, parses output, and analyzes failure patterns to inform the diagnostic stage.

LoopController: manages convergence detection and enforces maximum iteration limits to prevent infinite loops.

AutonomousCompiler: orchestrates the full loop, coordinating all components and maintaining provenance records for every modification.

15.2 AI-Powered Self-Writing Compilation

The SelfWritingCompiler extends autonomous compilation with AI assistance, following the sequence:

```
DESCRIBE &rrarr; AI-GENERATE &rrarr; COMPILE &rrarr; VERIFY &rrarr; TEST &rrarr;  
AI-DIAGNOSE &rrarr; AI-FIX &rrarr; RECOMPILE
```

The key components are:

AICLGenerator: creates complete AICL specifications from natural language task descriptions, including entities, behaviors, conditions, events, risks, recoveries, and validations.

AIDiagnoser: performs AI-powered root cause analysis and repair when the compilation loop encounters failures, proposing targeted fixes that are then applied and verified.

SelfWritingCompiler: the full loop with AI fallback, gracefully degrading to the deterministic autonomous loop when AI is unavailable.

15.3 Banking System: Full 10-Level Example

The 05_banking.aicl example demonstrates all 10 AICL levels in a single comprehensive specification. This example includes 9 entities (Customer, Account, Transaction, Card, Loan, Branch, CurrencyRate, FraudAlert, AuditEntry), 10 behaviors, 8 conditions, 8 events, 2 parallel groups, 4 optimization targets, 2 learning models, 4 encryption directives, 4 protection targets, native Python code blocks, and 14 validations. The example compiles to all 4 target languages, achieves 100% audit coverage, produces a valid Proof of Origin, and generates approximately 1,200 lines of code. It serves as both a demonstration of AICL's full capability and a stress test for the compilation pipeline.

15.4 Cryptographic Proof Signing

Cryptographic proof signing has been implemented in v2.0. Proof files can now be cryptographically signed using public-key cryptography, allowing third parties to verify not only the integrity of the proof (via SHA-256 hash binding) but also the identity of the compiler that produced it. Cross-compilation proof chains exist: when the same AICL specification is compiled to multiple target languages, the resulting proofs are linked by their shared source hash, enabling verification that all targets were derived from the same specification.

15.5 Runtime Provenance

A future direction is runtime provenance: extending the provenance chain from compile-time to runtime, so that every execution of generated code can be traced back to the architectural decision that produced it. This would transform AICL's Risk/Recovery pairs from compile-time specifications into runtime instrumentation, where the generated code includes monitoring hooks that detect risk conditions and trigger the corresponding recovery code. The provenance system would extend to cover runtime events, creating a complete trace from architectural risk identification through code generation to runtime recovery execution. This remains an open research question with significant implications for production monitoring and incident response.

15.6 Standardization

The No-Orphan Property and the Proof of Origin format are candidates for standardization. A position paper — "The No-Orphan Property: Towards Auditable Code Generation" — would argue that the No-Orphan Property should be considered a necessary condition for trustworthy code generation, not just a feature of one particular tool. The independent verifier provides a practical mechanism for enforcing this property, and the Proof of Origin format provides a standard artifact format that could be adopted by other code generation systems. Standardization would enable interoperability between different provenance-tracking compilers and independent verification tools.

16. Conclusion

AICL presents a new approach to the relationship between architecture and code. Rather than treating architecture as documentation that informs code, AICL treats architecture as the program itself, and compiles it into executable code with mandatory risk management, automatic test generation, full compilation provenance, and independently verifiable proof of origin.

The central contribution of this work has evolved through several stages. The original contribution was explicable compilation: the idea that a compiler should be able to explain why it generated every line of code, and that if it cannot explain, it should not generate. This was implemented through a provenance system that records every compilation decision and a deterministic compilation contract that makes provenance reproducible.

The contribution then deepened with auditable compilation: the principle that explainability is not enough — it must also be measurable. This was implemented through the audit system, which computes audit coverage and detects orphan artifacts, and through the formal No Orphan Property and Complete Coverage Property.

The current contribution is the Proof of Origin: the principle that auditability must not depend on trusting the compiler. The Proof of Origin is a self-contained, cryptographically bound artifact that can be verified by an independent third party using a 200-line script with zero AICL dependencies. This transforms the No Orphan Property from a feature of the compiler into a verifiable property of the artifact itself. This is a critical distinction: the property does not require you to trust AICL. It requires only that you can run a simple verification script.

The evolution of AICL reveals a trajectory that is more interesting than any single feature. The project began as an attempt to make a simpler language than C++. It became an exploration of explicable compilation, then auditable compilation, and now a code generation system with cryptographic proof of origin. Each stage discovered that the previous stage's endpoint was actually the beginning of a deeper problem. Explicable compilation raised the question of measurement. Measurement raised the question of trust. Trust raised the question of independent verification.

The latest stage of this evolution extends the principle to its logical conclusion: if the compiler can explain why it generated every line, it can also explain why it needs to fix a line, why a pattern should be learned, and why a specification should evolve. The autonomous compilation system — PatternLearner, SpecEvolver, and the Self-Writing Compiler — applies the same provenance-tracking discipline to the act of modification as the original compiler applied to the act of generation. Even when the code writes itself, every line has a reason, and that reason is recorded.

In a world where AI code generators produce code without accountability, where production systems fail at 3:00 AM because risks were documented but not compiled, and where the gap between architectural intent and implemented code grows wider every year, the principle of auditable compilation with proof of origin offers a different direction. Not more code generation, but more accountable code generation. Not faster compilation, but more transparent compilation. Not AI that writes code, but compilers that can prove why they wrote the code they did — and where that proof can be verified by anyone.

17. References

- [1] AICL Project Repository. <https://github.com/AFKmoney/AICL>
- [2] AICL Formal Grammar Specification. [spec/grammar.md](#)
- [3] AICL Compilation Provenance Implementation. [src/aicl/provenance.py](#)
- [4] AICL Independent Proof Verifier. [tools/verify_proof.py](#)
- [5] AICL Behavior Pattern Library. [src/aicl/patterns.py](#)
- [6] AICL Test Suite. [tests/test_aicl.py](#)
- [7] Garlan, D. and Shaw, M. Software Architecture: Perspectives on an Emerging Discipline. Prentice Hall, 1996.
- [8] Clements, P. et al. Documenting Software Architectures: Views and Beyond. Addison-Wesley, 2010.
- [9] Mernik, M. et al. 'Domain-Specific Languages: A Systematic Mapping Study.' Information and Software Technology, 2022.
- [10] Van Deursen, A. et al. 'Domain-Specific Languages: An Annotated Bibliography.' ACM SIGPLAN Notices, 2000.
- [11] Zig Programming Language. <https://ziglang.org/>
- [12] FIPS 180-4. Secure Hash Standard (SHA-256). NIST, 2015.
- [13] AICL Autonomous Compilation System. [src/aicl/autonomous.py](#)
- [14] AICL AI-Powered Code Generation. [src/aicl/ai_generator.py](#)
- [15] AICL AI Bridge (Node.js). [tools/ai_bridge.mjs](#)